ASSESSMENT TOOLS

Course Name: Theory of Computation **Course Code:** CSA1304

Course Outcome (CO5): Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. (BL: 3)

Assessment Tool: Design Critique Session

Question 1

A PDA is designed to recognize a^nb^n, but it fails for $a^nb^nc^n$. Critique the design and identify possible flaws in stack operations and state transitions. How would you improve it?

Answer:

How the original PDA works

The standard PDA for $L = \{a^nb^n \mid n \geq 0\}$ uses two phases controlled by two states: a 'push' state q0 that reads each 'a' and pushes a marker symbol A onto the stack (on top of an initial bottom-marker Z0), and a 'pop' state q1 that reads each 'b' and pops one A per b. The string is accepted if the input is exhausted exactly when the stack is back to Z0 (empty-stack or final-state acceptance). This works because the single stack performs exactly one comparison: it verifies that the count of a's equals the count of b's.

Why it fails for $a^nb^nc^n$

The language $\{a^nb^nc^n \mid n \geq 0\}$ requires two simultaneous count comparisons: (i) number of a's must equal number of b's, and (ii) that same count must also equal the number of c's. A single LIFO stack can only 'remember' one quantity at a time and can only be consulted once before it is emptied. Once the PDA pops all the A markers to match the b's, the stack is empty by the time the c's begin — there is no information left in memory to check the c-count against the original n. This is not a bug that can be patched; it is a structural limitation of pushdown memory, and it is exactly why $a^nb^nc^n$ is provably not a context-free language (this can be shown formally with the CFL pumping lemma).

Specific flaws in stack operations and state transitions

- Stack exhaustion problem: the stack is fully depleted after the b-phase, so no residual information survives into the c-phase.
- Only two states/phases exist (push-on-a, pop-on-b); there is no third phase or second independent counter to validate the c's.
- A single stack can encode only one nested/matching relationship (LIFO comparison of two symbol classes), not three mutually dependent counts.
- Any attempt to 'reuse' the same stack for the second comparison (c's) fails because information about n was destroyed the moment the corresponding A was popped.

How it could be improved

Because $a^n b^n c^n$ is inherently outside the power of any single-stack PDA (no restructuring of states or stack alphabet can fix this), a true fix requires more computational power than a PDA offers:

- Use two stacks (a 2-PDA), which is computationally equivalent to a Turing Machine and can independently retain the count for both comparisons.
- Use a Linear Bounded Automaton or a full Turing Machine, which can mark off a's, b's, and c's on tape without destroying earlier information.
- If only an approximate/partial acceptance is acceptable, the PDA can be redesigned to just accept a^n b^n (correctly) and reject anything with c's, but this changes the language being recognized rather than 'fixing' the original design.

The key critique to record is conceptual: the flaw is not a fixable implementation error but a demonstration of the boundary between Context-Free Languages and Context-Sensitive Languages in the Chomsky hierarchy.

Question 2

A student claims that the Context-Free Grammar E → E + T \| T, T → T * F \| F, F → (E) \| id can be directly converted into a Deterministic PDA (DPDA). Critically evaluate this statement with justification and suggest corrections if needed.

Answer:

Evaluating the claim

The claim is only partially correct, and it depends heavily on which CFG-to-PDA construction is used. There are two very different constructions in the theory of PDAs:

- The standard 'grammar substitution' construction (used to prove CFL = PDA-acceptable languages) simulates leftmost derivations by pushing the right-hand side of a production whenever a nonterminal is on top of the stack. This construction is inherently nondeterministic, because when more than one production applies to the same nonterminal (e.g., E → E + T | T), the PDA must guess which one to expand. This always yields an NPDA, never a DPDA, regardless of the grammar.
- The 'shift-reduce parsing' construction, used in practice for compilers, builds an LR-parsing table and uses the parser's stack as the PDA stack. This construction produces a DPDA if and only if the grammar is LR(1) (deterministic, unambiguous, and resolvable with one token of lookahead).
-

Applying this to the given grammar

The grammar shown (E → E + T | T; T → T * F | F; F → (E) | id) is the classical unambiguous, left-recursive expression grammar used in most compiler textbooks. It is unambiguous and is in fact LR(1)/LALR(1) — this is exactly the grammar used to build shift-reduce parsers for arithmetic expressions in tools like yacc/bison. So a DPDA does exist for the language it generates, and it can be built via the LR-automaton/shift-reduce method.

However, the statement 'CFGs can be directly converted into a DPDA' is false as a general claim: it is not true for every CFG. Only CFGs that are unambiguous and LR(k) generate a Deterministic Context-Free Language (DCFL), which is the exact class of languages recognizable by a DPDA. Many CFGs (e.g., ambiguous grammars, or grammars for inherently ambiguous languages) cannot be converted to any DPDA no matter how the construction is performed, because DPDA-recognizable languages (DCFLs) are a strict subset of CFLs recognized by general NPDAs.

Suggested correction

The corrected statement should be: 'This specific grammar can be converted into a DPDA because it is unambiguous and LR(1), and the conversion must use an LR/shift-reduce style construction — not the naive substitution-based grammar-to-PDA algorithm, which always produces an NPDA.' In general, only grammars generating Deterministic Context-Free Languages admit a DPDA; left recursion alone does not disqualify a grammar from being LR(1), but the naive top-down substitution method is unsuitable for deterministic parsing regardless of the grammar.

Question 3

A Turing Machine is designed using multiple tracks to solve a problem. Suggest alternative programming techniques like checking-off symbols or subroutines to do subtraction.

Answer:

Why multi-track designs are used, and their trade-off

A multi-track TM divides each tape cell into several parallel tracks (effectively enlarging the tape alphabet to $\Sigma \times \Sigma \times \ldots \times \Sigma$), which makes it convenient to store related data — for example, one track for operand m, another for operand n, and a third as a scratch/marker track. Multi-track TMs are not more powerful than single-track TMs (they are provably equivalent, since a k-track machine can always be simulated by a single-track machine with a larger alphabet); they simply make certain designs easier to describe. The trade-off is extra implementation/alphabet complexity.

Alternative 1 — Checking-off symbols technique

For subtraction $m - n$ (with $m \geq n$), represent both numbers in unary on a single track separated by a delimiter, e.g., 0^m # 0^n. The machine repeatedly:

- Scans to the leftmost unmarked 0 in the m-block and replaces it with a marker symbol x (checked off).
- Moves right to the leftmost unmarked 0 in the n-block and replaces it with x as well (checked off).
- Repeats this left-to-right sweep, checking off one symbol from each block per pass, until every 0 in the n-block has been checked off.

- The 0's remaining unchecked in the m-block are exactly m − n, which becomes the result after clean-up (erasing the x's and the n-block).

This technique avoids the need for a second track entirely — the 'bookkeeping' that a track would have provided is instead done in place by overwriting symbols, at the cost of extra left-right head movement.

Alternative 2 — Subroutine (procedural) technique

Subtraction can also be built out of smaller, reusable TM 'subroutines', much like a program calls functions:

- A decrement subroutine that, given a unary block of 0's, moves to its rightmost symbol and erases it (reducing the block's count by 1), then returns control to a designated 'return state'.
- A counting/loop control mechanism: to compute m − n, invoke the decrement subroutine on the m-block, then decrement a counter representing n; repeat this pair of calls n times using a loop implemented with states that branch back to the start of the subroutine until the n-counter reaches zero.
- Each subroutine has its own clearly defined entry state and exit/return state(s), and the main program's states simply transfer control to the subroutine and resume from where they left off, exactly like a call/return pattern in ordinary programming.

Comparison and recommendation

Both alternatives achieve subtraction on a single track without requiring the extra alphabet complexity of a multi-track design, and neither changes the computational power of the machine (by Church–Turing equivalence, single-track, multi-track, checking-off, and subroutine-based TMs all recognize exactly the recursively enumerable languages). The checking-off method is simpler to describe for straightforward counting/matching tasks, while the subroutine method scales better when subtraction needs to be reused as a building block inside a larger TM program (e.g., as part of multiplication or division), since it promotes modular design over ad-hoc symbol marking.

Question 4

While designing a PDA that accepts the language L = {w | w ∈ (a + b)* and na(w) = nb(w)}, acceptance is defined only by final state, ignoring empty stack acceptance. Critique this design choice. Would it affect the language recognized? Explain with reasoning.

Answer:

Theoretical background: final-state vs. empty-stack acceptance

It is a standard theorem of PDA theory that the class of languages accepted by final-state PDAs is exactly the same as the class accepted by empty-stack PDAs — both equal the class of Context-Free Languages. There is a well-known construction to convert any empty-stack PDA into an equivalent final-state PDA (and vice versa): a new bottom-of-stack marker and a new dedicated accepting state are added, and the machine transitions to that accepting state precisely when the original empty-stack condition would have been met. So, in principle, choosing final-state acceptance instead of empty-stack acceptance does not reduce the class of recognizable languages, and na(w) = nb(w) remains a context-free language under either acceptance mode.

The practical critique of this specific design choice

The theoretical equivalence only holds if the conversion/construction is done correctly. The real risk in this design is not the choice of final-state acceptance itself, but whether the final states are placed at the correct point in the stack configuration:

- If the PDA is built so that entering a final state is deliberately tied to the stack top being exactly the bottom marker Z0 (i.e., the net balance of a's and b's is genuinely zero at that point), then final-state acceptance is perfectly correct and equivalent to empty-stack acceptance.
- If, however, the final states are placed carelessly — for example, marked as accepting simply because the input has been fully read, without checking that the stack has returned to Z0 — the PDA could incorrectly accept unbalanced strings that still have leftover a's or b's sitting on the stack.
- Conversely, if the PDA is only willing to reach an accepting state when the stack is completely empty (not just at Z0) but the design never actually pops the bottom marker, it could wrongly reject strings that are genuinely balanced.

Would it affect the language recognized?

Not in theory — a correctly constructed final-state PDA recognizes exactly the same language as its empty-stack counterpart, since CFLs are closed under this equivalence. In practice, however, if the acceptance condition is not carefully tied to the stack content (i.e., checking that the top of stack equals the base marker, indicating equal counts), the specific PDA as designed could accept a different — and incorrect — language than intended. The correction is straightforward: gate every transition into a final state on the stack top being the bottom marker Z0, guaranteeing that 'accepted' truly coincides with 'na(w) = nb(w)'.

Question 5

A system designer suggests replacing a Turing Machine with a PDA to reduce computational complexity. Critically compare FA, PDA, and Turing Machine capabilities and evaluate whether this replacement is valid.

Answer:

Comparing the three models

Feature	Finite Automaton (FA)	Pushdown Automaton (PDA)	Turing Machine (TM)
Memory	None (only finite states)	One stack (LIFO, unbounded)	Infinite tape, random read/ write
Language class	Regular languages	Context-Free Languages	Recursively Enumerable Languages
Example limitation	Cannot count (e.g., a^nb^n)	Cannot handle multiple independent counts (e.g., $a^nb^nc^n$) or arbitrary access	No inherent limitation — can simulate any algorithm
Chomsky hierarchy position	Type 3 (Regular)	Type 2 (Context-Free)	Type 0 (Unrestricted)

The hierarchy of power is strict: Regular languages ⊂ Context-Free Languages ⊂ Context-Sensitive Languages ⊂ Recursively Enumerable Languages, matching FA ⊂ PDA ⊂ Linear Bounded Automaton ⊂ Turing Machine.

Evaluating the proposed replacement

The suggestion to replace a Turing Machine with a PDA 'to reduce computational complexity' is not valid in general. The two machines are not interchangeable substitutes — a PDA's stack only allows last-in-first-out access to its own memory, whereas a TM's tape allows unrestricted random-access reading and rewriting anywhere, any number of times. Because of this, many

computations a TM performs are provably impossible for any PDA: languages requiring more than one independent count or comparison ($a^nb^nc^n$), languages requiring matching two identical halves of arbitrary strings (ww), and general-purpose algorithmic computation (arithmetic, sorting, simulation of other machines) all fall outside what any PDA can recognize or compute, regardless of how it is designed.

When the replacement would be reasonable

- If the actual task the system needs to perform is provably context-free in nature (e.g., matching nested brackets/parentheses, validating simple balanced structures, parsing programming-language syntax), then a PDA is not just adequate but preferable — it is simpler, uses less memory, and has well-understood linear-time parsing algorithms (e.g., LL/LR parsing).
- If the task requires unrestricted computation, arbitrary memory access, or checking constraints beyond simple nested matching, a PDA cannot substitute for a TM at all — the replacement would not 'reduce complexity', it would make certain problems entirely unsolvable by the new system.

Conclusion

The replacement is valid only for the strict subset of problems that are inherently context-free; it is invalid as a general strategy for reducing 'computational complexity', because a PDA is a strictly weaker model of computation than a Turing Machine, not merely a faster or lighter version of the same thing. The correct engineering decision is to match the model to the language class of the actual problem, not to substitute a weaker automaton purely for efficiency.